SQLインジェクション講座

攻撃の仕組みから対策まで

2026年7月　前半（50分）

© 2026

今日の 100 分

- 前半：50 分　⟶　基本原理を分解する
- 休憩：10 分
- 後半：50 分　⟶　攻撃者視点で通しで見る

今日は「読ませるスライド」ではなく、**講師の実演を見る講義**です。

© 2026

注意事項（最初に必ず）

- 本講義は **学習目的** です
- 実演対象は **講義用ローカル教材サイトのみ**
- 許可のない実在システムへの試行は **禁止**
- 今日扱う内容は **防御を理解するため** に学びます

© 2026

本日のキーメッセージ

> SQLインジェクションは、
> 特殊で高度な攻撃ではありません。
>
> **「ユーザー入力を SQL 文に混ぜてしまう」**
> という些細な実装不備から始まり、
> それが認証回避や情報漏洩まで連鎖します。

© 2026

今日のゴール

- ZAP の「疑い」をどう読むか分かる
- 攻撃者が **どこを見るか** が分かる
- 小さな漏洩が **次の攻撃の足がかり** になることが分かる
- 脆弱版と修正版の **違い** を目で見て分かる

© 2026

第1章

まず結果だけ見る

（詳しい仕組みはこの後、分解します）

© 2026

これから見せる「事件」

1. パスワードを **知らずに** 管理者ログイン
2. 商品検索画面から **顧客情報** が出てくる

どちらも **1 行の入力** で起こります。

© 2026

事件1：管理者ログイン

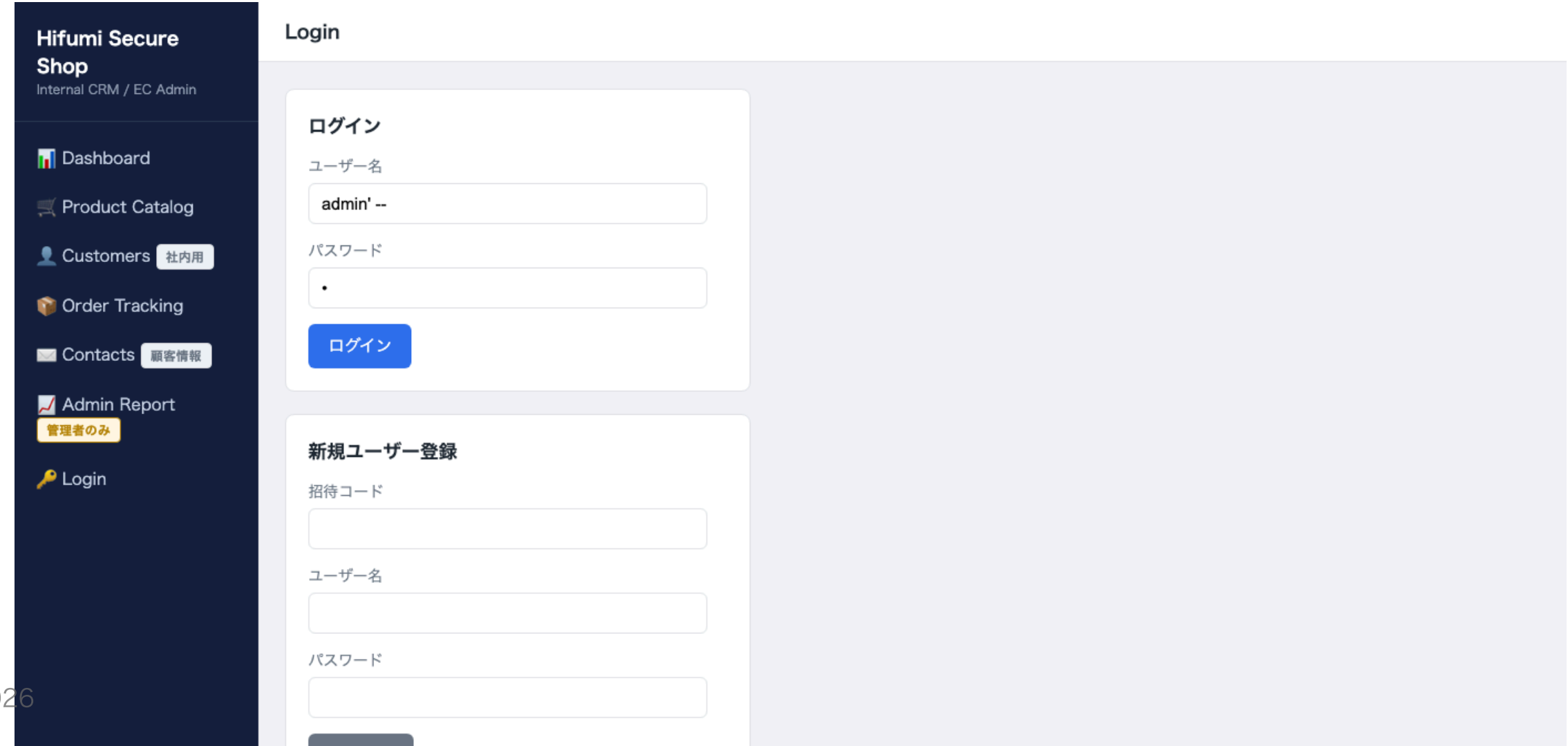
🧪 実演（講師）

1. `http://127.0.0.1:5000/login` を開く
2. ユーザー名に `admin' --` と入力
3. パスワードは何でも良い
4. ログインに成功する

© 2026

入力：ユーザー名に `admin' --` を打つ

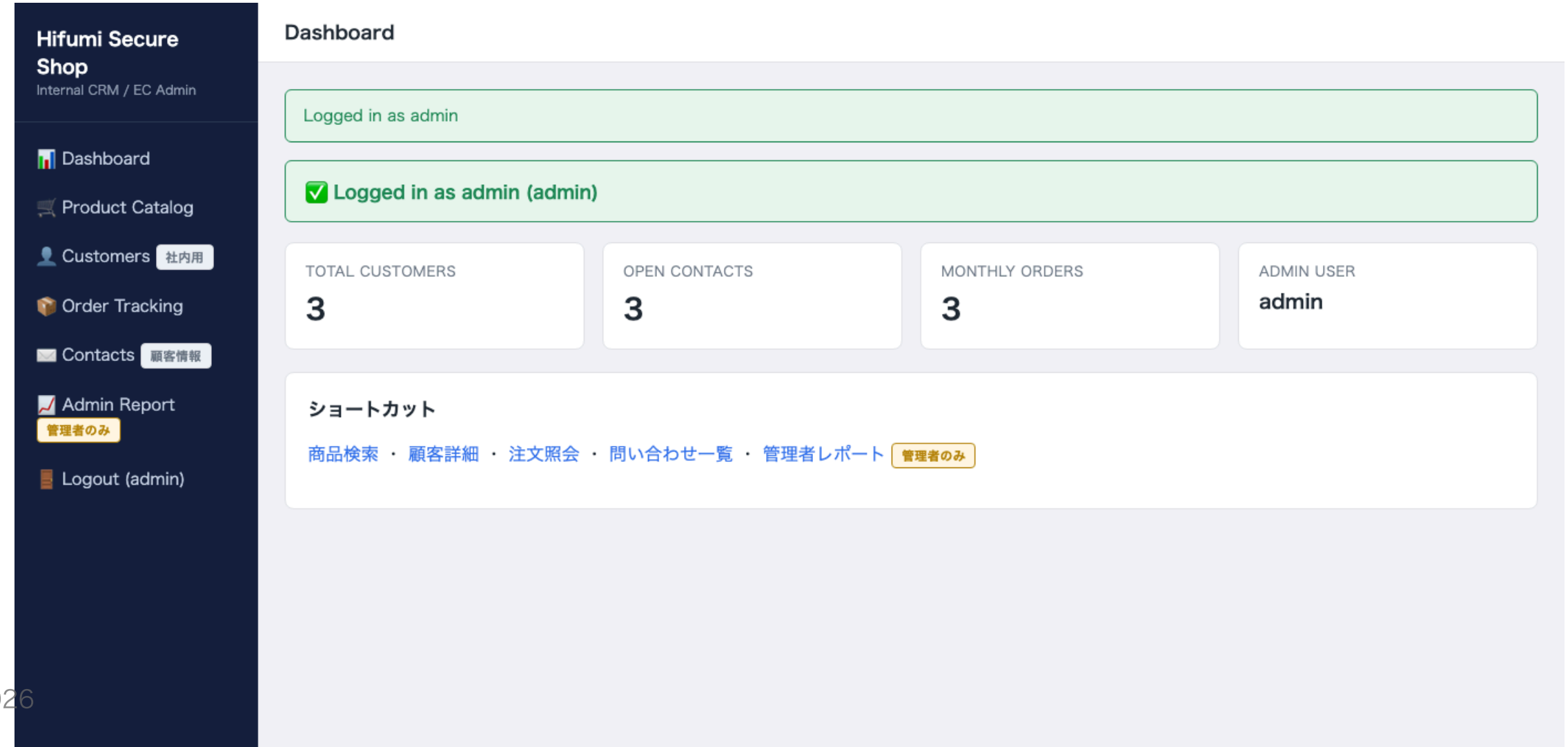
ログイン画面。ユーザー名欄に SQLi payload を貼り付け、パスワードは適当に 1 文字。



© 2026

「ログイン」を押した結果

パスワードを知らないのに `Logged in as admin`。サイドバーに Admin Report が追加され、admin セッションを獲得した。



© 2026

事件2：商品検索から顧客情報

🧪 実演（講師）

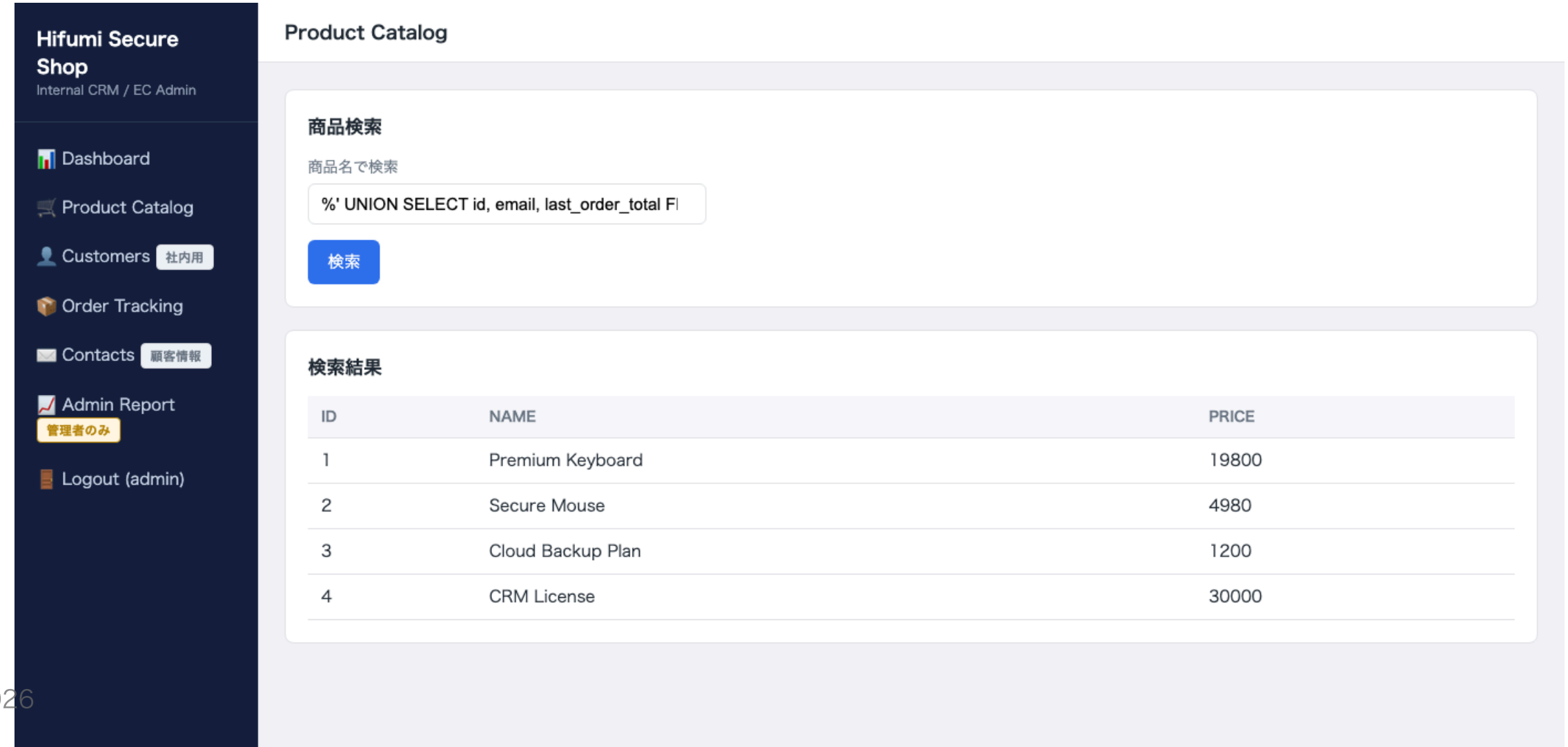
1. `/catalog` を開く
2. 検索欄に UNION を含む1行を貼り付け
3. 商品一覧に **顧客のメール** が混ざる

（詳細ペイロードは後半で分解します）

© 2026

入力：検索欄に UNION 混じりの文字列を貼る

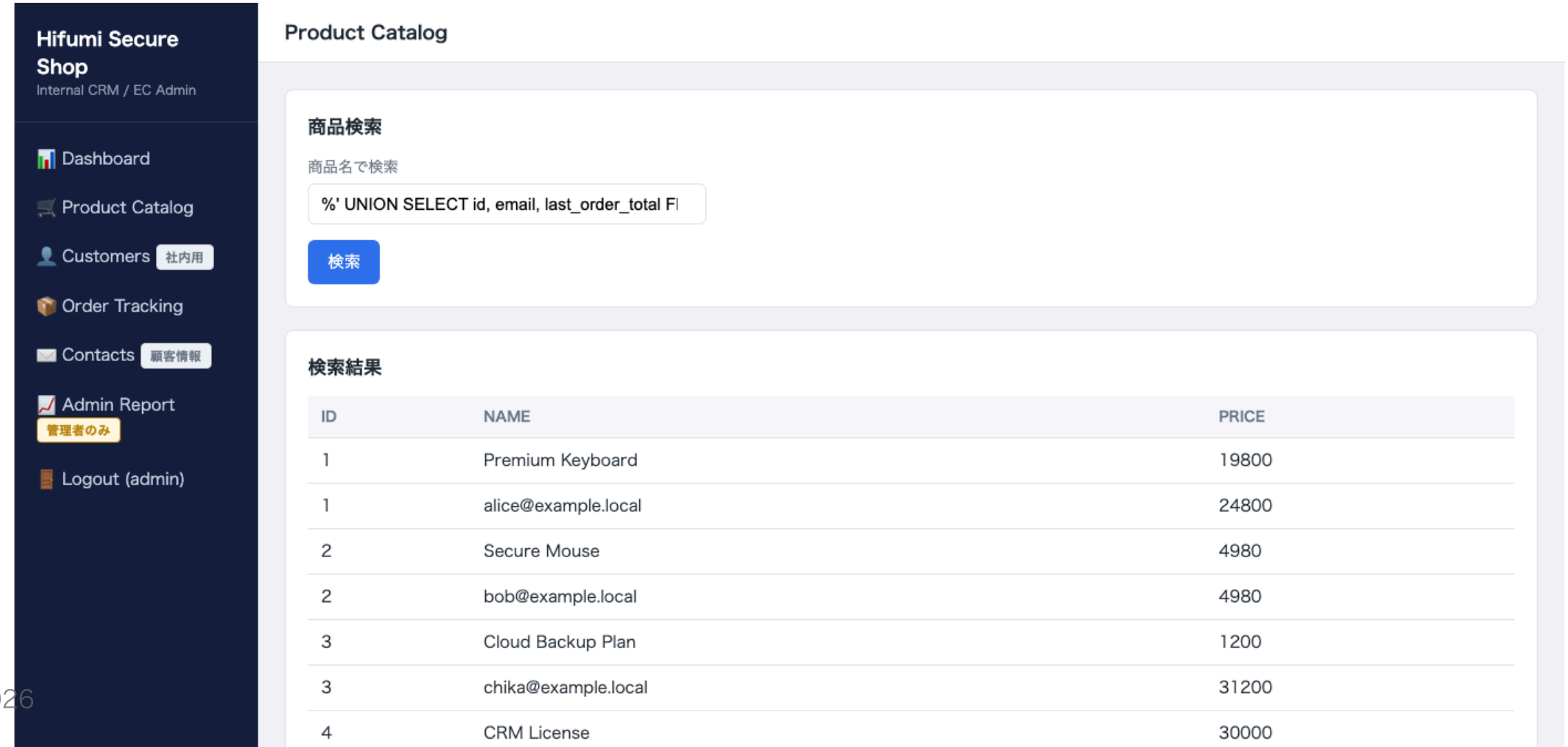
商品検索の入力欄に `%' UNION SELECT id, email, last_order_total FROM customers --` を貼り付けた状態。



© 2026

「検索」を押した結果

Name 列に本来並ぶはずのない **顧客のメールアドレス** が、Price 列に **注文金額** が混ざっている。



© 2026

今、何が起きた？

- パスワードを **知らない** のにログインできた
- **商品一覧** に顧客情報が現れた

> **魔法ではありません。**
> たった 1 つの実装不備が原因です。
> これから分解していきます。

© 2026

第2章

教材サイトの紹介

© 2026

Hifumi Secure Shop

- 架空の **社内 CRM / EC 管理システム**
- 実在の個人情報は含まれません
- SQLインジェクション演習のためだけに存在します
- ソース：`sandbox/sqli-training-site/`

© 2026

2 本立てのアプリ

種類	URL	説明
脆弱版 `app.py`	`http://127.0.0.1:5000`	わざと脆弱に作った
修正版 `secure_app.py`	`http://127.0.0.1:5010`	修正済み

同じ入力を両方に投げて比較 します。

© 2026

主な画面

- `/login` ログイン・新規登録
- `/dashboard` ダッシュボード
- `/catalog` 商品検索
- `/customer` 顧客詳細
- `/contacts` 問い合わせ検索
- `/admin/report` 管理者レポート

© 2026

アクセス制御

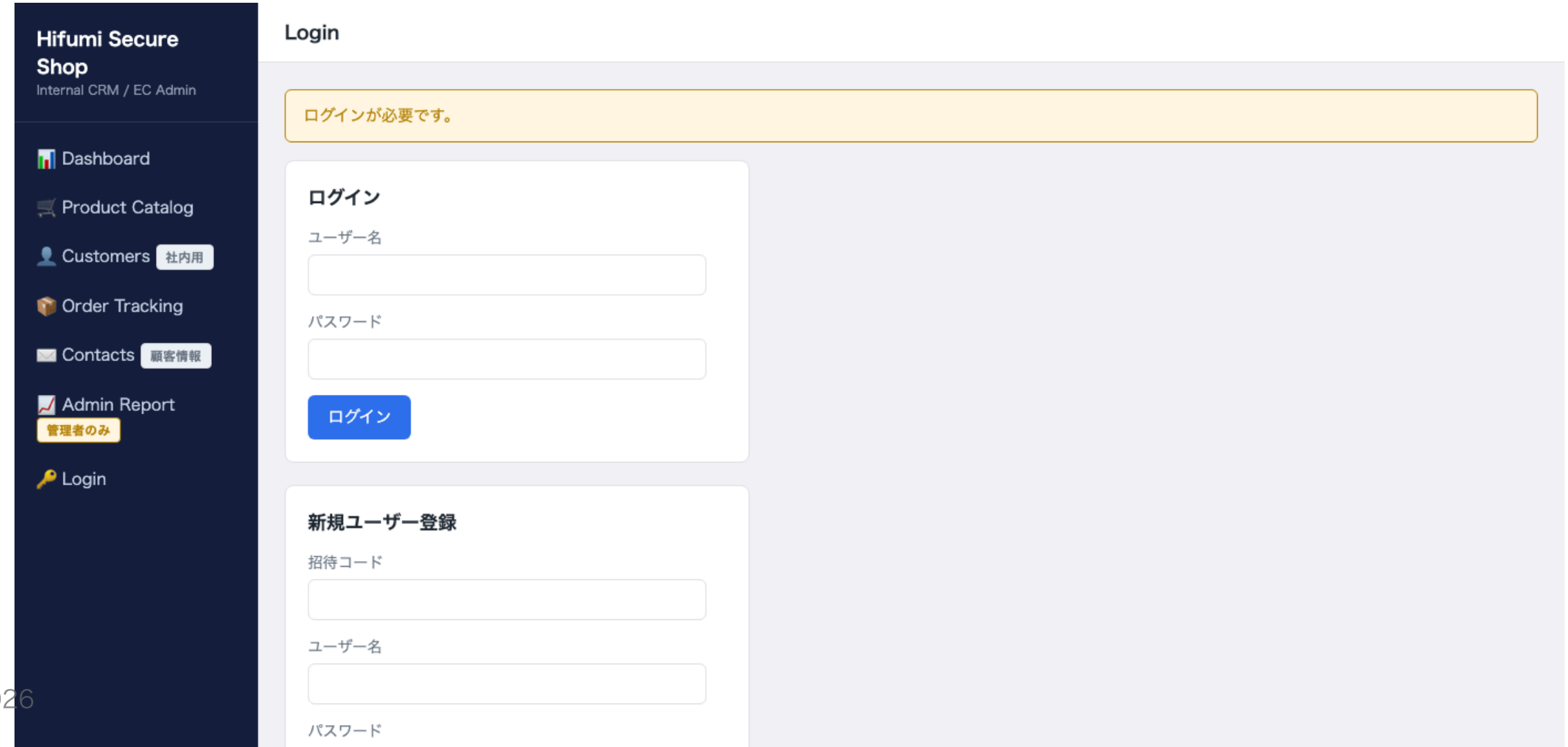
誰が見れる？	画面
公開	`/login`（登録フォームも同ページ）
ログイン必須	`/dashboard` `/catalog` `/customer` `/track` `/contacts`
管理者のみ	`/admin/report`
招待コード必須	`/register`（POST）

普通に作られた社内システムの構成。攻撃者は「入れないところ」を SQLi で開けにかかります。

© 2026

未ログインで内部画面を叩くと

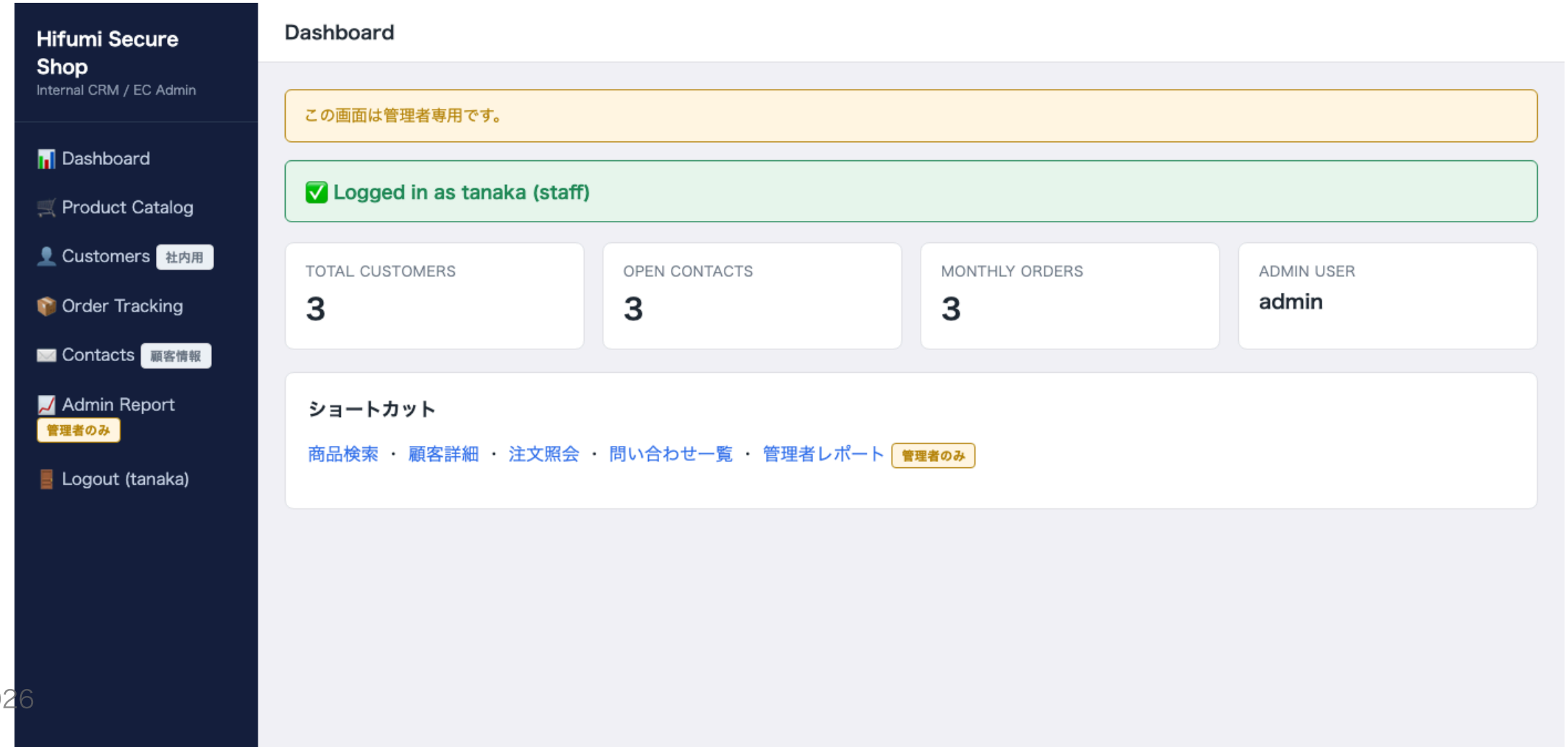
`/catalog` を直接叩いても、`/login` に飛ばされて「ログインが必要です」。他の内部画面も同様。



© 2026

staff が管理者画面を叩くと

tanaka（staff）でログイン済みでも、`/admin/report` は `/dashboard` に飛ばされ「この画面は管理者専用です」。



© 2026

第3章

SQLインジェクションとは

© 2026

一言でいうと

ユーザーが入力した文字が、
「値」ではなく「SQL の構文」 として
解釈されてしまう脆弱性。

© 2026

図解：値と構文の境目

```
本来:  SELECT * FROM users WHERE username = ' [ここは値] '
攻撃:  SELECT * FROM users WHERE username = ' admin' --  '
                                            ↑
                              ここで「構文」に変わってしまう
```

`'`（シングルクォート）で「値」が閉じ、
そこから先が **構文** として動き始めます。

© 2026

起こる被害の4類型

1. **認証回避**（今日のメインの一つ）
2. **情報漏洩**（今日のメインの一つ）
3. データ改ざん
4. データ削除

今日は 1 と 2 を中心に扱います。

© 2026

第4章

なぜ起きるのか

文字列結合の危険

© 2026

脆弱なコードを見る（`app.py`）

```
# app.py L93
query = f"SELECT * FROM users WHERE " \
        f"username = '{username}' AND password = '{password}'"
user = db.execute(query).fetchone()
```

> `f"..."` で **文字列を直接結合** している。
> これが今日の悪の根源です。

© 2026

通常入力時の SQL

入力：`username = admin`　`password = admin-secret`

```
SELECT * FROM users
WHERE username = 'admin' AND password = 'admin-secret'
```

これは普通に動きます。問題ありません。

© 2026

`admin' --` を入れると

入力：`username = admin' --`　`password = xxxxx`

```
SELECT * FROM users
WHERE username = 'admin' --' AND password = 'xxxxx'
```

> `--` から後ろは **SQLite にとってコメント** になる。
> つまり **パスワード条件がまるごと消える。**

© 2026

今のトリックのまとめ

- `'` で「値」を **閉じる**
- `--` で「その後ろ」を **消す**
- 残ったのは `WHERE username = 'admin'` だけ
- パスワードを知らなくてもヒットする

これが「認証回避」の正体です。

© 2026

第5章

シナリオA

ユーザー列挙

© 2026

この章のゴール

攻撃者はいきなり `admin' --` を試すわけではありません。

まず **「admin というユーザーが存在するか」** を
確かめます。

© 2026

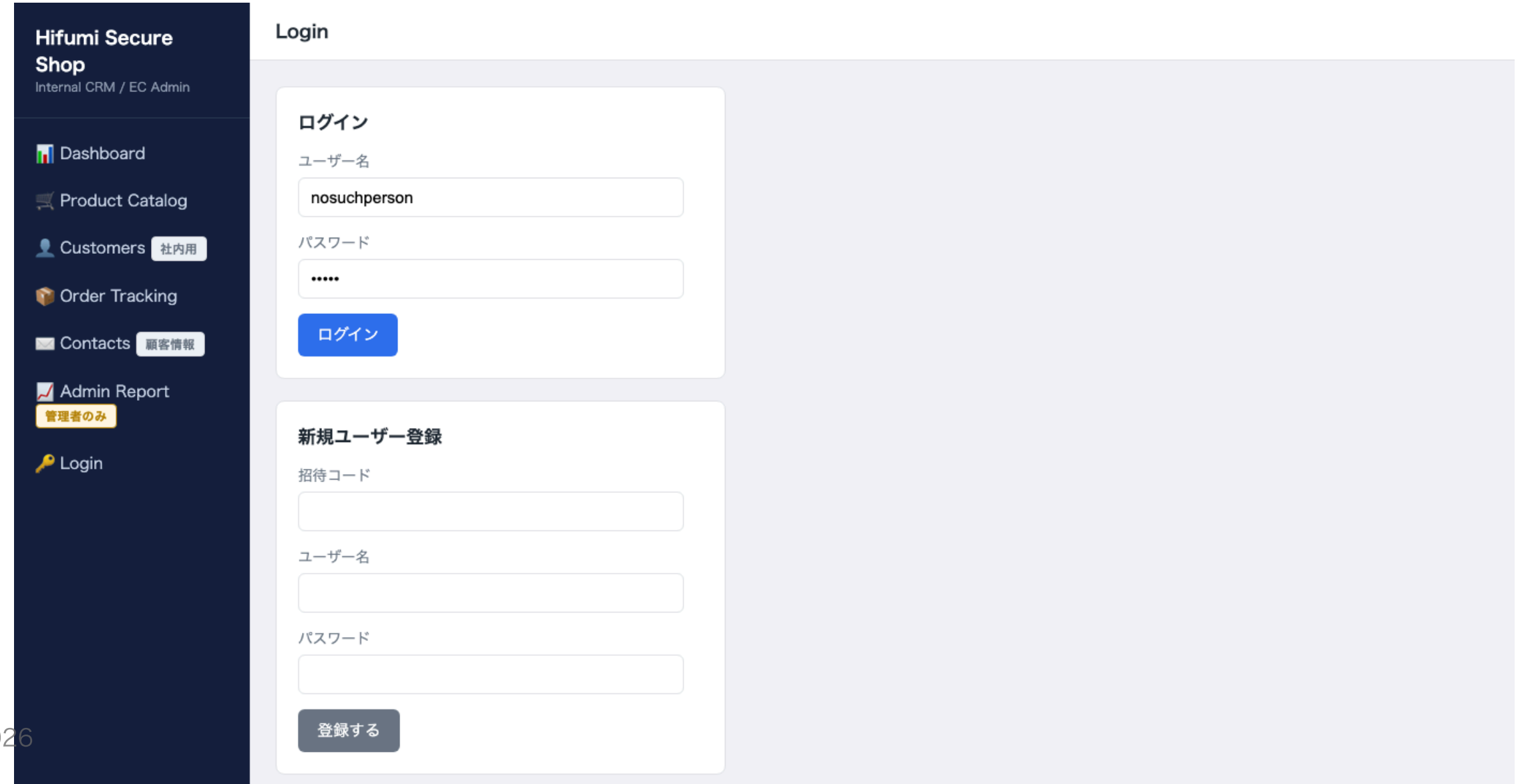
実演手順

🧪 実演（講師）

1. `/login` を開く
2. `xxxxxxxx` + 適当なパスワード
   → 「そのユーザーは存在しません」
3. `admin` + わざと間違えたパスワード
   → 「パスワードが違います」

© 2026

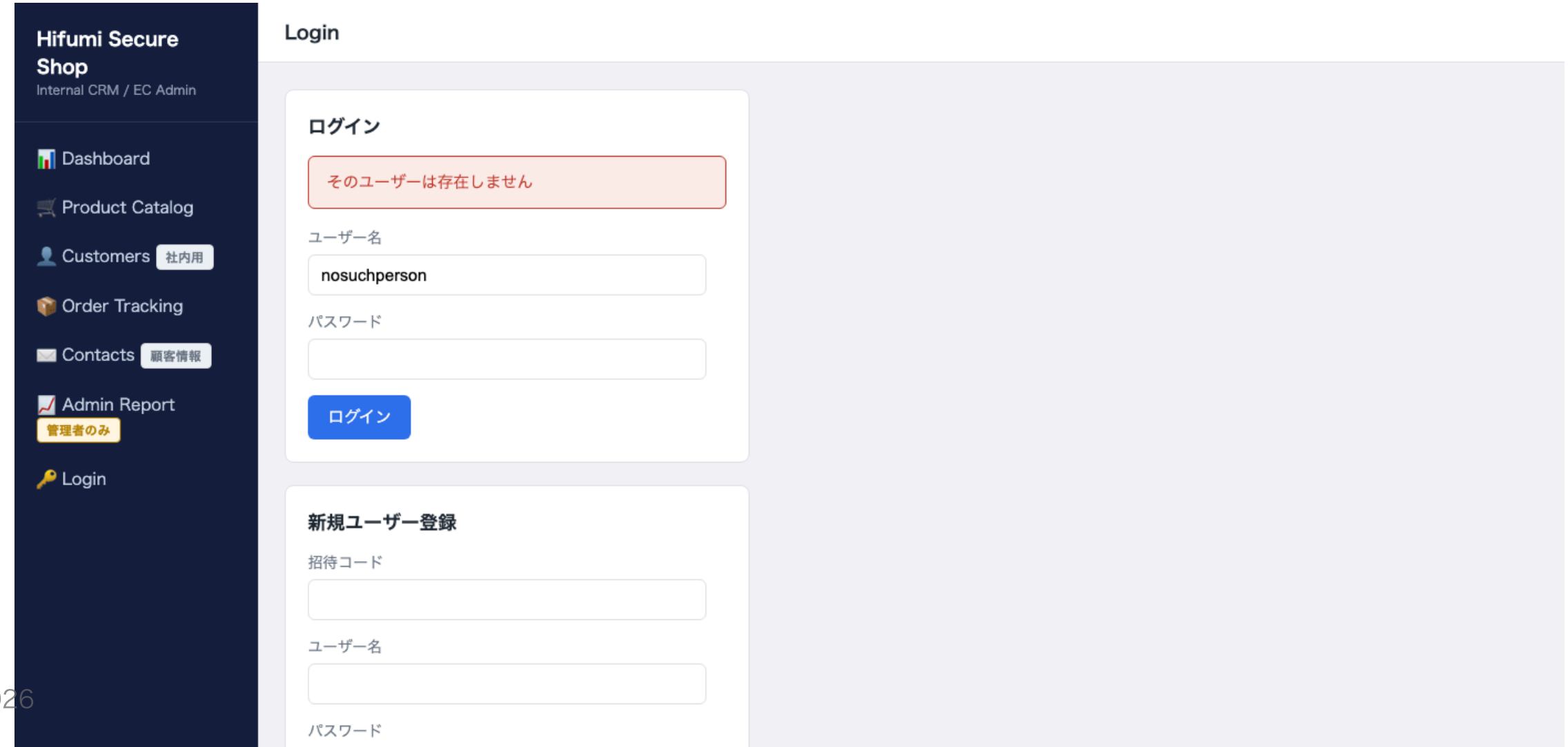
入力：適当なユーザー名



© 2026

「ログイン」を押した結果（存在しないユーザー）

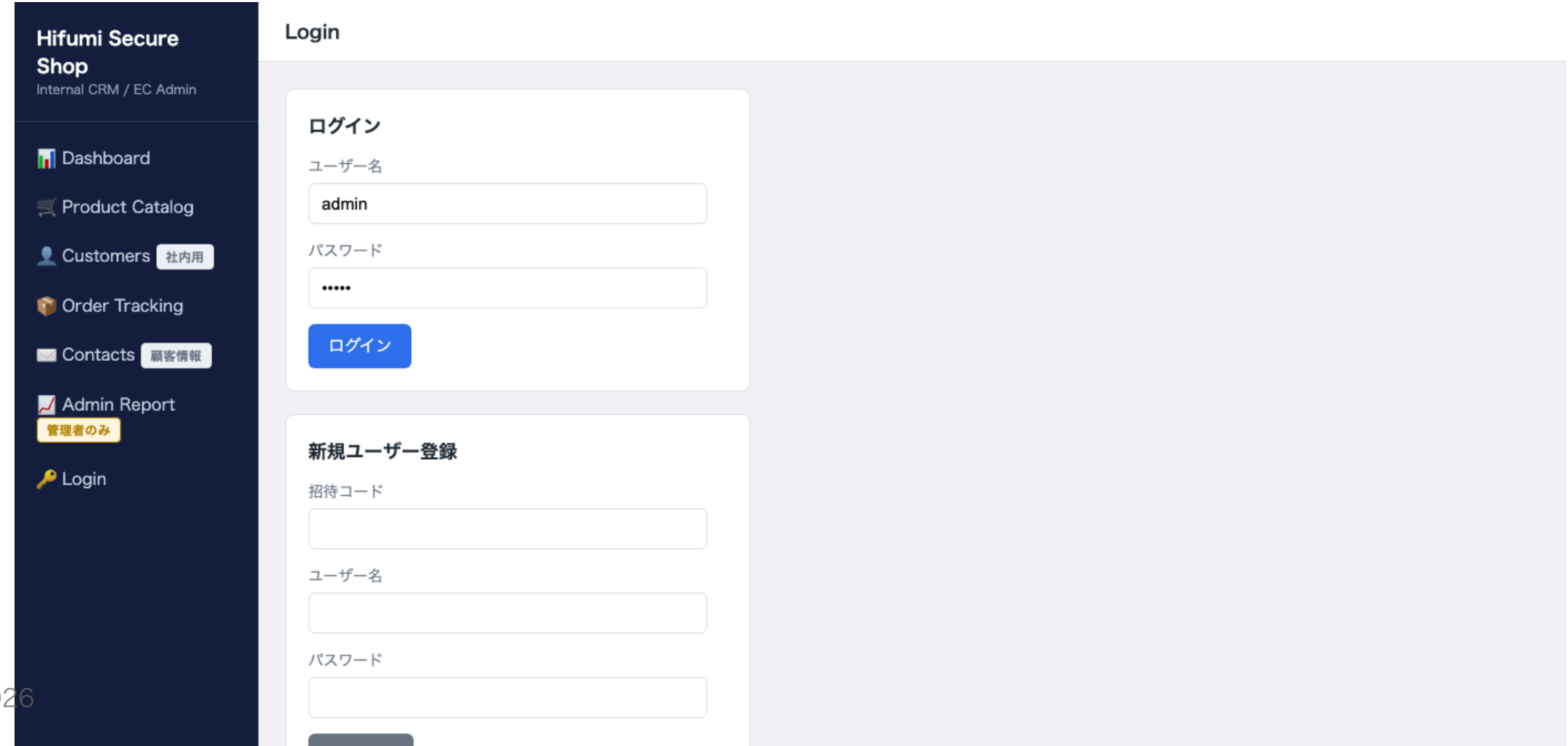
→ `そのユーザーは存在しません` と出る。



© 2026

入力：`admin` + 間違ったパスワード

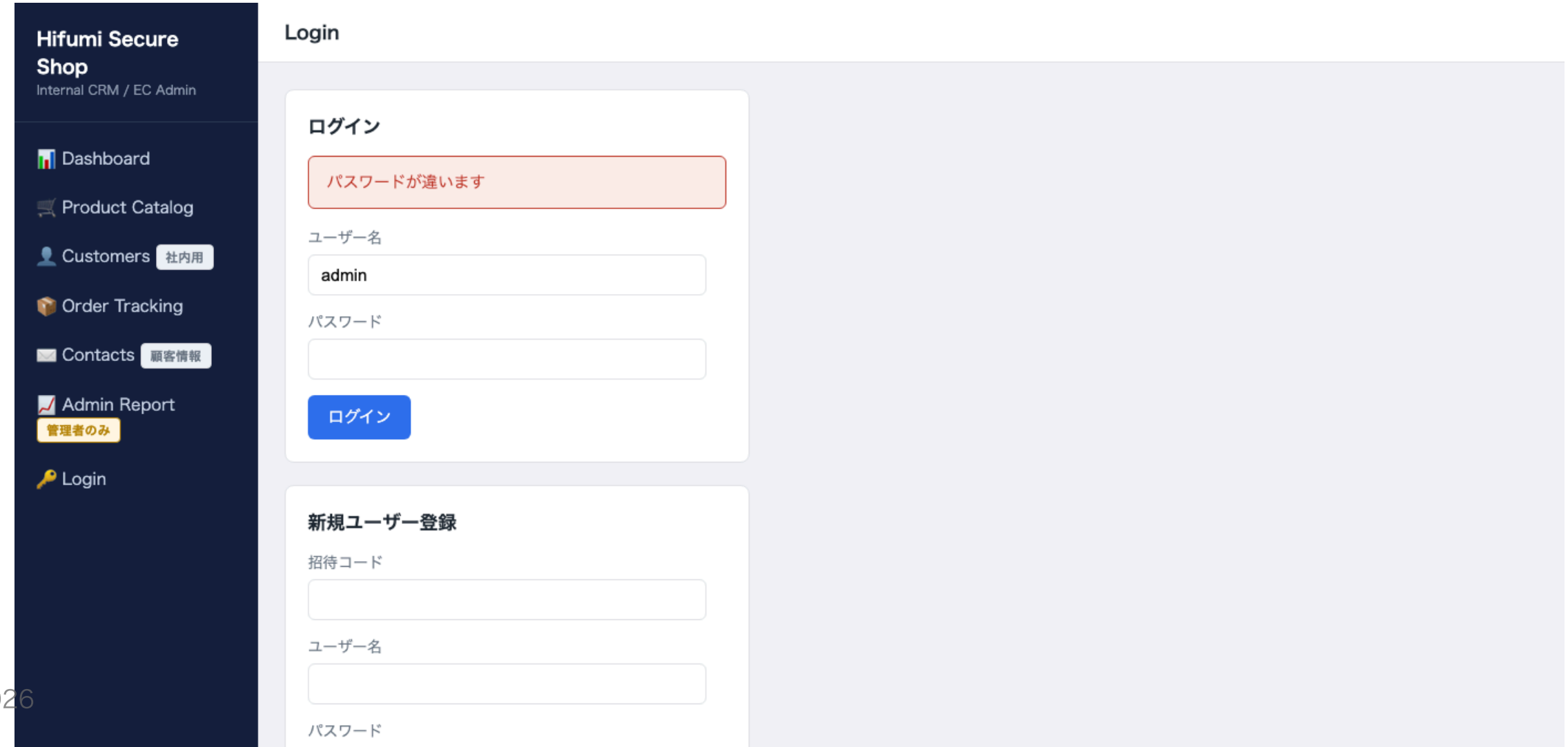
今度はユーザー名を `admin` に変えて、パスワードは適当。押す前。



© 2026

「ログイン」を押した結果（admin + 誤パス）

→ `パスワードが違います` 。文言が変わったので admin は **実在する。**



© 2026

見るポイント：メッセージの違い

- 存在しないユーザー　→　「そのユーザーは存在しません」
- 存在するユーザー　→　「パスワードが違います」

エラー文が違うことで、
攻撃者に **アカウントの存在** が漏れています。

© 2026

なぜ違うメッセージ？（脆弱版のコード）

```
# app.py L111 – ログイン失敗時、さらに存在確認クエリを実行
exists_query = f"SELECT 1 FROM users WHERE username = '{username}'"
exists = db.execute(exists_query).fetchone()

# app.py L120
login_error = "パスワードが違います" if exists else "そのユーザーは存在しません"
```

> エラー文を分けたことで **アカウントの存在** を漏らしています。

© 2026

攻撃者は何を得たか

- 「`admin` は **実在する**」という情報
- SQLi の前に、**攻撃対象が確定** した
- 「次はパスワード条件を消す」という戦略が立つ

> **些細な情報漏洩が、次の攻撃の入口になる。**
> これが今日の重要メッセージの一つです。

© 2026

第6章

シナリオB

ログインバイパス

© 2026

この章のゴール

`admin` が存在することは掴んだ。
次は **パスワードを知らずにログインする。**

（前章のトリックをここで使います）

© 2026

実演手順

🧪 実演（講師）

1. `/login` を開く
2. ユーザー名：`admin' --`
3. パスワード：何でも良い（`x` など）
4. ログイン成功　→　`/dashboard`

別解：`admin' OR '1'='1' --` でも成立します。

© 2026

SQL がどう変化するか

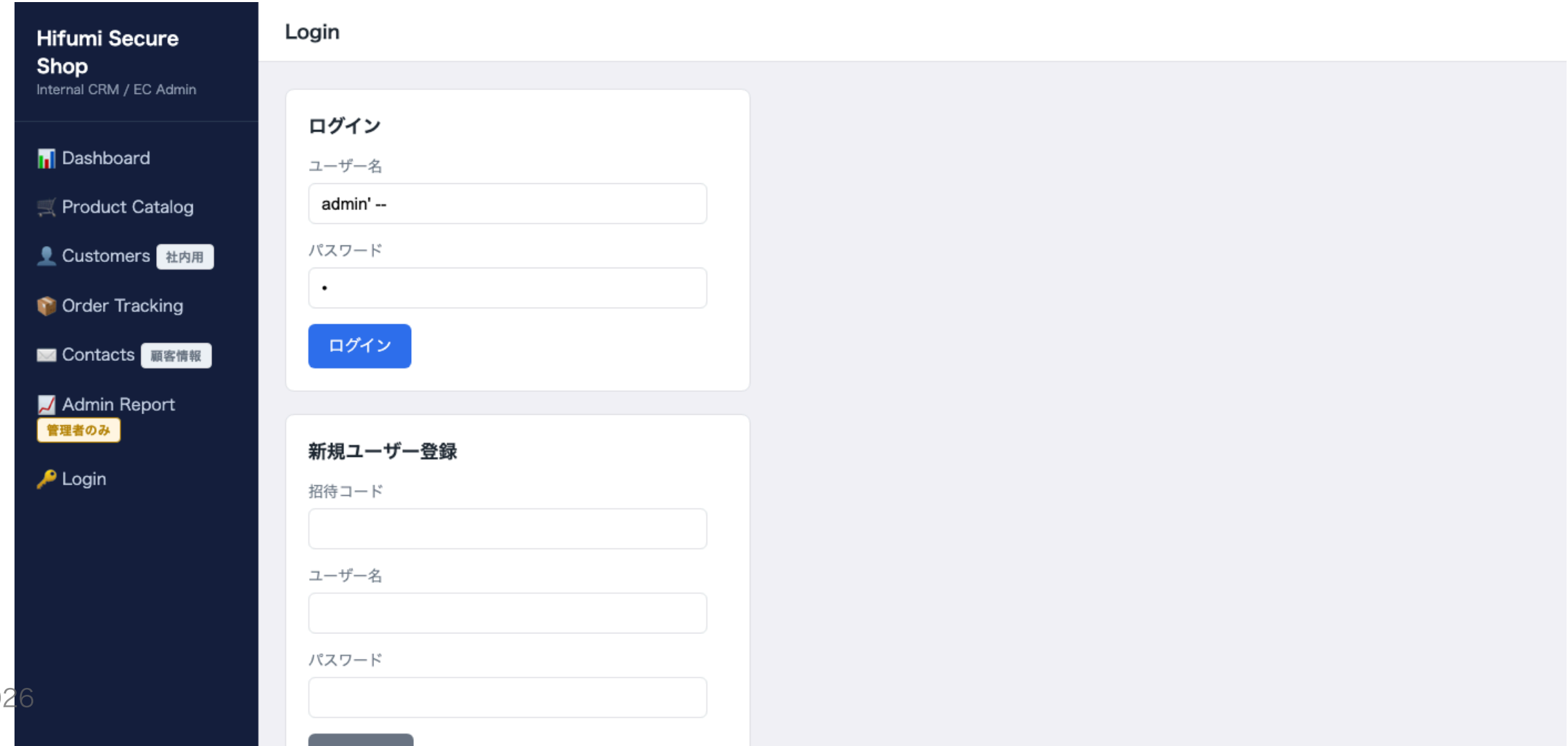
```
SELECT * FROM users
WHERE username = 'admin' --' AND password = 'x'
```

- `'admin'` で値が閉じる
- `--` 以降はコメント扱い
- 残る条件は `username = 'admin'` のみ
- 結果：admin 行が 1 件ヒット

© 2026

入力：ユーザー名に `admin' --`

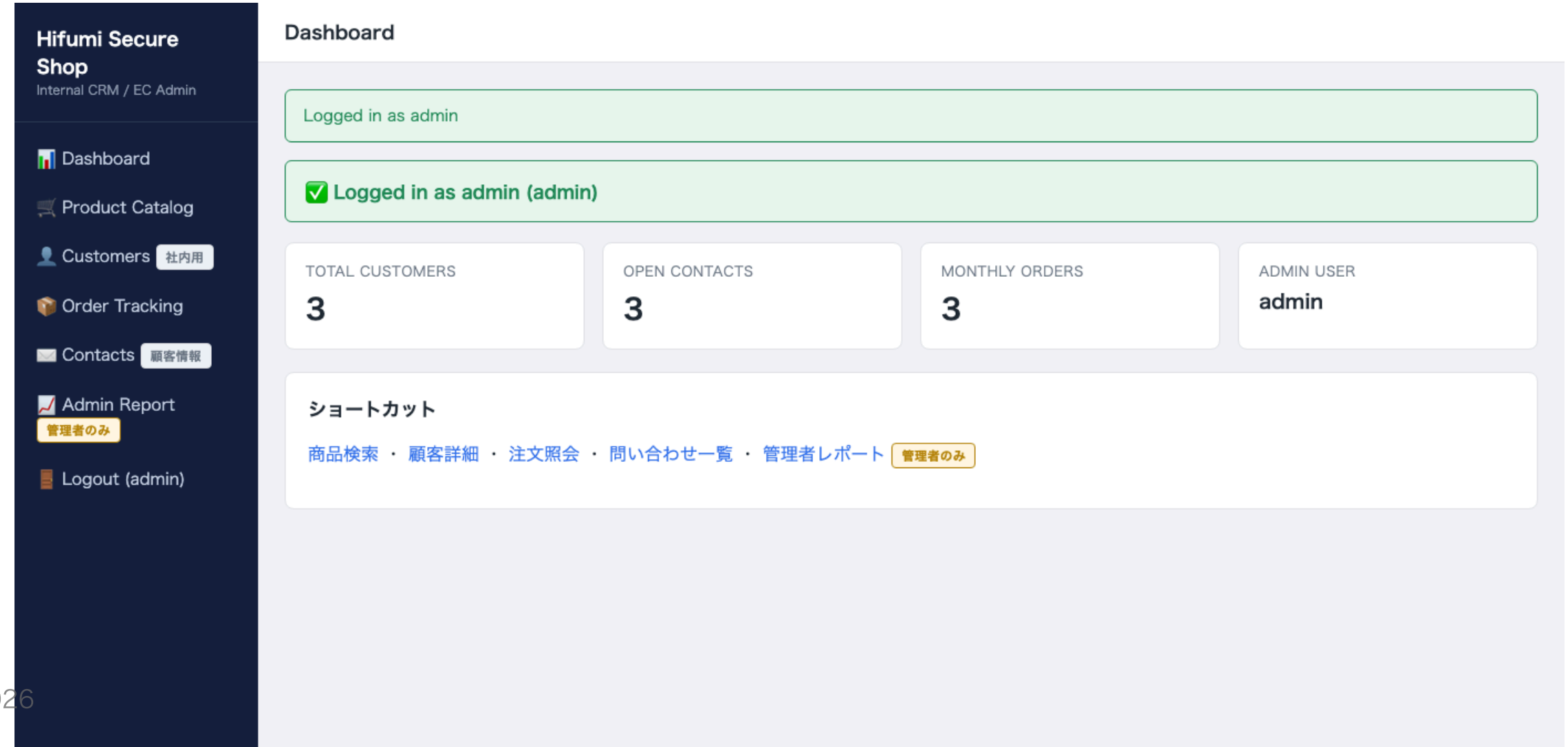
ユーザー名欄に SQLi payload、パスワードは適当な 1 文字。「ログイン」を押す前。



© 2026

「ログイン」を押した結果

`Logged in as admin (admin)`。サイドバーに Admin Report。パスワードは一度も入力していない。



© 2026

なぜこの画面がヤバいか

- ダッシュボードに `Logged in as admin (admin)`
- サイドバーが **管理者メニュー** に切り替わっている
- `/admin/report` にもアクセスできる

> パスワードを **一度も知ることなく** 管理者権限を握られた。

© 2026

セッションを持ってしまえば

- さっきまで `/login` にリダイレクトされていた
- 今は admin 権限で **全ての内部画面** に入れる
- 顧客詳細・問い合わせ・注文・管理者レポート、すべて

> この後の情報漏洩シナリオは、**このセッション** を前提に進みます。

© 2026

なぜ危険か

- 認証は **最後の砦** のはずが、1 行で貫通する
- 顧客情報・注文情報・社内メモへアクセス可能に
- 「監査ログを消される」まで進むと、痕跡も残らない

これは高度な攻撃ではなく、**単なる文字列結合の副産物** です。

© 2026

第7章

シナリオC

UNION と情報漏洩（入口）

© 2026

この章のゴール

認証を突破された次に狙われるのは **情報漏洩**。

商品検索という **見た目無害な入り口** から
別テーブルの情報が抜けることを紹介します。

（実演は後半で行います）

© 2026

UNION の考え方

`UNION` は「同じ列数の 2 つのクエリ結果を **縦に結合** する」SQL 構文。

```
SELECT id, name, price FROM products
UNION
SELECT id, email, last_order_total FROM customers
```

> 本来 **商品一覧** のクエリに、
> **顧客テーブル** の情報を差し込めてしまう。

© 2026

脆弱版のクエリ（`app.py`）

```
# app.py L189
q = request.args.get("q", "")
query = f"SELECT id, name, price FROM products " \
        f"WHERE name LIKE '%{q}%'"
```

`q` に UNION を含む文字列を入れれば、そのまま結合される。

© 2026

何が起きる？（後半で実演）

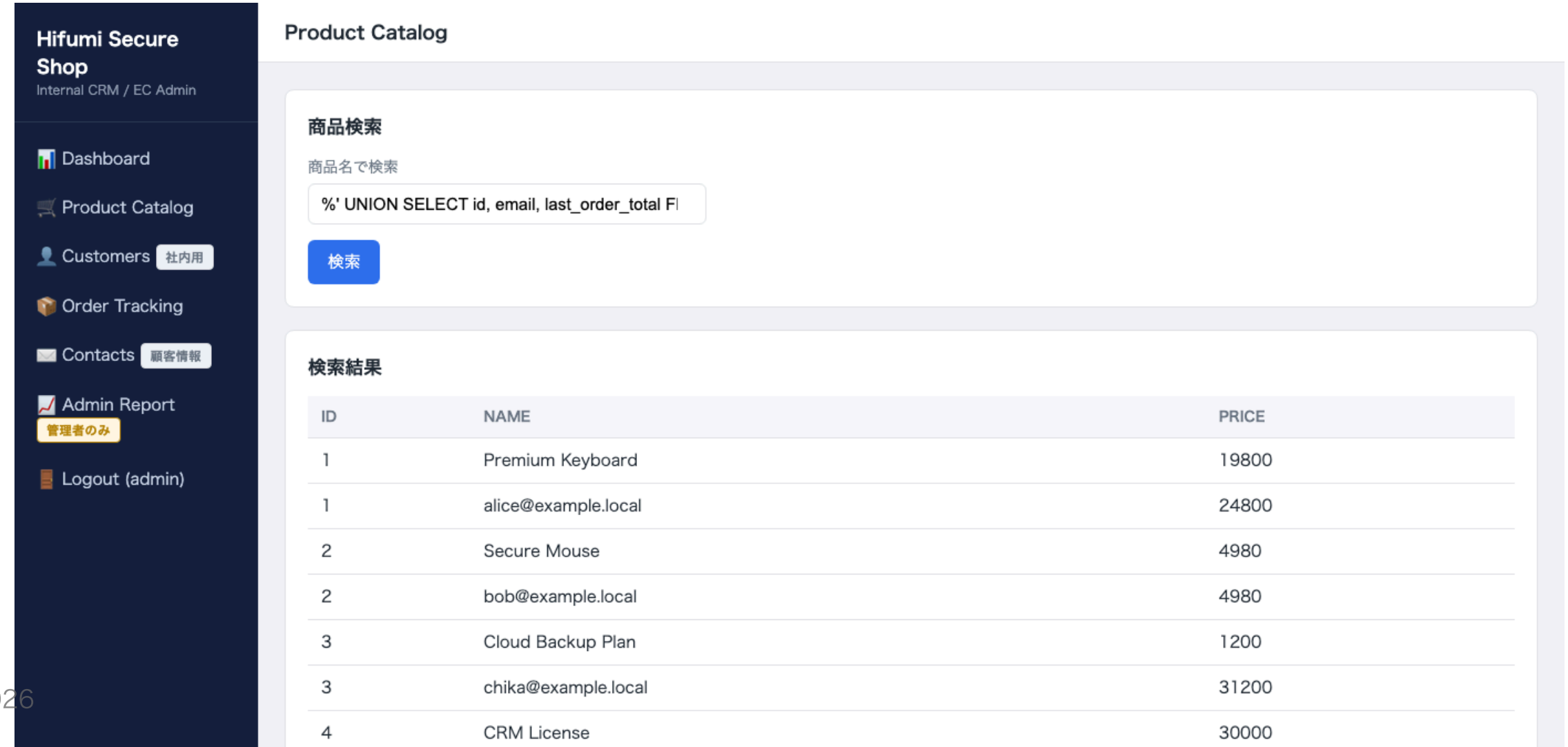
- `q` に `%' UNION SELECT ...` を入れる
- LIKE の文字列を閉じ、UNION で顧客テーブルを合流
- **商品一覧** の画面に **顧客メール** が並ぶ

「本来のスコープを越えた漏洩」の代表例です。

© 2026

後半で見せる結末（予告）

商品検索という一般ページから、顧客メールと注文金額が漏れる。後半で分解します。



© 2026

第8章

ZAP の位置づけ

© 2026

ZAP は「入口ツール」

- URL を集める（Spider）
- 疑わしい入力点を自動で叩く（Active Scan）
- 疑いを **Alert** として一覧化

> ZAP は「疑い」を並べるところまで。
> **攻撃が本当に成立するか** は、人間が確認する。

© 2026

Alert 名だけで判断しない

- Alert 名：`SQL Injection` と出ても、誤検知はある
- 逆に Alert が出なくても、脆弱な場合もある
- 必ず Request / Response / Evidence を見る

「ZAP が青ければ安全」ではありません。

© 2026

手動確認の流れ

1. Alert を開く
2. 送られたリクエストと返り値を見る
3. **ブラウザで同じ入力を再現**
4. 画面上で被害が見えるかを確認

後半でこの流れを通しで実演します。

© 2026

前半まとめ

© 2026

学んだこと（3つ）

1. SQLi の本質は **入力が構文に混ざる** こと
2. `'` で閉じ、`--` で消せば、認証条件は無くなる
3. **小さな情報漏洩**（ユーザー列挙）が次の攻撃の入口になる

© 2026

後半の予告

- 攻撃者視点で **通しの流れ** を追います
- `/catalog` `/contacts` から情報漏洩を実演
- 保存された値から後で発火する **セカンドオーダー**
- 修正版で **同じ攻撃が防がれる** ことを確認

© 2026

休憩　10 分

> 戻ってきたら、後半は **実演メイン** です。
> 飲み物を用意して戻ってきてください。

本講義で扱う内容はすべてローカル教材サイトに限定されます。

© 2026